Lab Report No: 02

Course: Computer Programming, Second Semester

Date: 28/02/2026

Student Name: Irshad Ahmad

Etea Roll No: 5124858

Registration No: BF25NWELE0699

Instructor: Muhammad Rizwan

Program 01 – Empty Program

Objective:

To understand the fundamental structure of a C++ program without performing any operations.

To learn the role of the main() function as the entry point of any C++ program.

To observe compilation and execution of a basic program.

Apparatus:

Computer with C++ compiler installed.

Text editor or IDE (Code::Blocks, Dev-C++, or Visual Studio).

Console/terminal for output.

Procedure:

1. Open the IDE or text editor.
2. Create a new C++ source file.
3. Write the program skeleton including #include<iostream> and int main().
4. Compile and run the program.
5. Observe that nothing is displayed on the console.

Theory:

main() is mandatory as the entry point of C++ programs.

#include<iostream> provides the input-output functionality.

return 0; signals successful termination of the program.

Empty programs are used to test setup and compiler correctness.

Program:

```
#include<iostream>

using namespace std;

int main() {

return 0;

}

// Nothing will be printed
```

Conclusion:

Program runs successfully without output.

Helps understand basic program structure.

Program 02 – Hello World

Objective:

To display a message on the screen.

To introduce console output using cout.

To familiarize students with basic syntax and line breaks using endl.

Apparatus:

Computer with C++ compiler.

IDE or text editor.

Console for output.

Procedure:

1. Open IDE and create a new C++ file.
2. Include the iostream library and use main() function.
3. Use cout to display "Hello World" with endl.
4. Compile and run the program.
5. Observe the message on the console.

Theory:

cout outputs text to the console.

<< operator directs output to console.

endl inserts a newline character.

Hello World is the first step for beginners to check environment setup.

Program:

```
#include <iostream>

using namespace std;

int main() {

cout << "Hello World" << endl;

return 0;

}
```

Conclusion:

Successfully displays Hello World.

Introduces basic console output.

Program 03 – Variable Display (int/float)

Objective:

To learn about variables and data storage.

To understand difference between integer and floating-point numbers.

To display variable values using cout.

Apparatus:

Computer with C++ compiler.

IDE/text editor.

Console output.

Procedure:

1. Open IDE and create a new C++ file.
2. Declare a float or integer variable.
3. Assign value and use cout to print it.
4. Compile and run the program.
5. Observe the displayed value.

Theory:

Variables store data in memory.

int stores whole numbers; float stores decimal numbers.

Printing variables verifies correctness and data types.

Variable usage is essential for calculations and data manipulation.

Program:

```
#include <iostream>

using namespace std;

int main() {

float x = 60.98546;

cout << "x = " << x << endl;

}
```

Conclusion:

Correctly displays the variable.

Demonstrates data types and console output.

Program 04 – User Input (Age)

Objective:

To learn how to accept user input during program execution.

To practice combining input, processing, and output.

To introduce interactive programming.

Apparatus:

Computer with C++ compiler.

IDE or text editor.

Console for input/output.

Procedure:

1. Open IDE and create a new C++ file.
2. Declare variable to store input.
3. Prompt user using cout.
4. Take input using cin.
5. Display input using cout.
6. Compile and run the program.

Theory:

cin takes input from the keyboard.

Variables store the input for processing or output.

Interactive programs allow dynamic user responses.

Input validation is key for real applications.

Program:

```
#include <iostream>

using namespace std;

int main() {

int age;

cout << "Dear user kindly enter your age: ";

cin >> age;

cout << "Your age is = " << age << endl;

return 0;

}
```

Conclusion:

Takes input and displays it correctly.

Demonstrates basic I/O interaction.

Program 05 – Print Name Multiple Times

Objective:

To understand loops and repetition.

To automate repetitive tasks.

To avoid manual writing of multiple statements.

Apparatus:

Computer with C++ compiler.

IDE/text editor.

Console output.

Procedure:

1. Open IDE and create a C++ file.

2. Write a for loop to repeat output.

3. Use cout to print the name inside the loop.

4. Compile and run the program.

Theory:

Loops allow repeated execution of code.

for loop is used when iteration count is known.

Looping saves time and reduces errors.

Loops are foundational for advanced algorithms and automation.

Program:

```
#include <iostream>

using namespace std;

int main() {

for(int i=0; i<10; i++)

cout << "My Name is Irshad Ahmad" << endl;

return 0;

}
```

Conclusion:

Prints the name 10 times.

Demonstrates use of loops for repetition.

Program 06 – Check Positive or Negative Number

Objective:

To determine if a number is positive or negative.

To learn conditional statements (if-else).

To strengthen decision-making skills in programming.

Apparatus:

Computer with C++ compiler.

IDE/text editor.

Console for input/output.

Procedure:

1. Open IDE and create a new file.

2. Prompt user for integer input.

3. Use if-else to check positive or negative.

4. Display result.

5. Compile and run the program.

Theory:

if-else statements allow conditional execution.

Comparison operators (>=, <) determine program flow.

Logic is fundamental to real-world problem solving.

Program:

```
#include <iostream>
using namespace std;
int main() {
int Number;
cout << "Enter an integer: ";
cin >> Number;
if(Number>=0)
cout << "Your number is positive = " << Number << endl;
else
cout << "Your number is negative = " << Number << endl;
return 0;
}
```

Conclusion:

Checks sign accurately.

Demonstrates conditional statements.

Program 07 – Loop Print Name 100 Times

Objective:

To demonstrate large iterations using loops.

To reinforce automated repetition and loop counters.

Apparatus:

Computer with C++ compiler.

IDE/text editor.

Console output.

Procedure:

1. Open IDE and create a new file.

2. Write for loop from 0 to 100.

3. Print name inside loop.

4. Compile and run.

Theory:

Loops repeat code for a set number of iterations.

Loop counters control execution accurately.

Efficient repetition reduces manual coding.

Program:

```
#include <iostream>

using namespace std;

int main() {

for(int i=0; i<=100; i++)

cout << "irshad ";

return 0;

}
```

Conclusion:

Prints “irshad” 101 times.

Demonstrates loop-based repetition.

Program 08 – While Loop Counter

Objective:

To learn condition-based repetition using while loops.

To practice counting numbers sequentially.

Apparatus:

Computer with C++ compiler.

IDE/text editor.

Console output.

Procedure:

1. Open IDE and create a new file.

2. Initialize counter variable.

3. Use while loop to check condition.

4. Print value and increment counter.

5. Compile and run.

Theory:

while loops repeat code as long as condition is true.

Counter variables prevent infinite loops.

Useful when number of iterations is unknown.

Program:

```
#include <iostream>

using namespace std;

int main() {

int i=1;

while(i<=5){

cout << i << endl;

i++;

}

return 0;

}
```

Conclusion:

Counts 1 to 5 successfully.

Demonstrates condition-based iteration.

Program 09 – Sum of Two Numbers

Objective:

To calculate sum of two numbers entered by the user.

To practice arithmetic operations and I/O in one program.

Apparatus:

Computer with C++ compiler.

IDE/text editor.

Console output.

Procedure:

1. Open IDE and create a new file.

2. Prompt user for two numbers.

3. Store numbers in variables.

4. Calculate sum and display result.

5. Compile and run.

Theory:

Arithmetic operations like addition use variables.

cin takes input; cout displays output.

Programs performing calculations are fundamental to learning logic.

Program:

```
#include <iostream>

using namespace std;

int main() {

int a, b, sum;

cout << "Enter first number: ";

cin >> a;

cout << "Enter second number: ";

cin >> b;

sum = a + b;

cout << "Sum of " << a << " and " << b << " is = " << sum << endl;

return 0;

}
```

Conclusion:

Calculates sum correctly.

Combines input, processing and output in one program